

动态规划课程期末大作业报告

Decision Transformer: Reinforcement Learning via Sequence Modeling

王志豪周开轩张沁怡

2026-06-15

目录

1	选题与项目目标	1
2	文献阅读与算法理解	2
2.1	研究背景	2
2.2	论文贡献	2
2.3	关键算法	2
3	实验环境	3
4	Baseline 与实验设置	3
5	真实实验结果	4
5.1	数据集统计	4
5.2	主结果: BC、RCBC、短历史 MLP 与 DT	4
5.3	Target Return Sweep	5
5.4	训练轮数分析	6
5.5	训练损失	7
6	原论文 D4RL 对齐尝试: Hopper	7
7	结果分析	8
7.1	为什么 RCBC 失败	8
7.2	为什么 DT 能优于 RCBC	8
7.3	局限性	8
8	结论	9
9	参考文献	9

1 选题与项目目标

本项目选择复现论文 **Decision Transformer: Reinforcement Learning via Sequence Modeling**。该论文由 Lili Chen 等人在 2021 年提出,研究方向属于 Offline Reinforcement Learning、Transformer 和 Sequence Modeling 的交叉领域。

项目	内容
英文题目	Decision Transformer: Reinforcement Learning via Sequence Modeling
中文题目	决策 Transformer: 通过序列建模实现强化学习
作者	Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, Igor Mordatch
年份	2021
论文链接	https://arxiv.org/abs/2106.01345
官方代码	https://github.com/kzl/decision-transformer

项目目标是在课程规模内复现 Decision Transformer 的两个核心机制：一是利用历史序列解决部分可观测任务中的记忆问题；二是通过 return-to-go 条件化生成不同回报水平的行为。

2 文献阅读与算法理解

2.1 研究背景

传统强化学习通常依赖 Bellman equation、value function、policy improvement 或 policy gradient 来优化策略。Offline RL 的约束在于智能体只能使用已有轨迹，不能继续探索环境；如果直接做 value learning，容易在数据分布外动作上出现过估计。

Decision Transformer 提出的核心问题是：能否不显式做 Bellman backup，而是把强化学习轨迹改写为条件序列建模问题，用 Transformer 直接预测动作？

2.2 论文贡献

论文的主要贡献包括三点。

第一，它将强化学习问题转化为 conditional sequence modeling。模型不再显式学习 value function，而是直接建模轨迹序列中的动作生成。

第二，它引入 return-to-go 作为条件输入。模型不仅看历史状态和动作，还接收目标回报，因此可以根据不同 target return 输出不同行为。

第三，论文在 Atari、OpenAI Gym/D4RL 和 Key-to-Door 等任务上验证了方法的有效性，表明 Transformer 的上下文建模能力在离线强化学习中具有实际价值。

2.3 关键算法

对一条长度为 T 的轨迹，Decision Transformer 定义 return-to-go：

$$\hat{R}_t = \sum_{t'=t}^T r_{t'}$$

模型输入序列为：

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots, \hat{R}_T, s_T, a_T)$$

训练时截取最近 K 个时间步，共 $3K$ 个 token。return-to-go、state、action 分别通过模态特定的 embedding 层，再加上 timestep embedding 后送入带 causal mask 的 Transformer。动作预测头根据 state token 对应的 hidden state 预测当前动作。

在本项目的离散动作设置下，训练目标为：

$$\max \log p(a_t \mid \hat{R}_{1:t}, s_{1:t}, a_{1:t-1})$$

测试时先给定 **target return**，模型根据目标回报、历史状态和历史动作自回归生成动作；环境执行动作后获得 **reward**，再更新剩余目标回报并继续决策。

3 实验环境

本项目最终采用 **MiniGrid MemoryS17Random** 作为主实验环境。

该任务要求智能体先在起点区域观察目标物体类型，随后沿走廊移动，并在岔路口选择与起点物体匹配的一侧。离开起点后，当前观测不再包含起点物体信息，因此任务本质上是一个部分可观测记忆问题。

为适应课程规模内的训练成本，本项目使用紧凑的部分观测特征，而不是直接使用原始图像和 **mission** 文本。

特征	含义
agent x/y	智能体当前位置，归一化
direction	智能体朝向，归一化
visible start object	起点物体仍在起点区域时使用 one-hot 编码；离开后为 [0,0]
in start room	是否仍在起点区域
visible fork objects	接近岔路口时显示上下两侧物体的 one-hot 编码
normalized step	当前步数归一化

4 Baseline 与实验设置

本项目比较五个模型。

模型	输入	作用
BC	当前 state	无条件行为克隆
RCBC	当前 state + return-to-go	条件化前馈 baseline
SBC-K3	最近 3 步 state	短历史 MLP baseline
SBC-K5	最近 5 步 state	更长短历史 MLP baseline
DT	return-to-go + state/action history	论文核心序列模型

离线数据由两类脚本策略混合得到。

数据来源	比例	行为
explore policy	35%	走到随机一侧，约一半成功、一半失败
expert policy	65%	读取环境内部目标，仅用于生成离线数据，走到正确一侧

正式实验设置如下。

配置	数值
seeds	0, 1, 2
每个 seed 离线轨迹数	400
epoch	40
batch size	128
context length	20
Transformer 层数	1
embedding dim	96
attention heads	4
learning rate	1e-3
每次评估 episode	100

5 真实实验结果

5.1 数据集统计

seed 0 的离线数据统计如下。

指标	数值
episodes	400
transitions	4044
平均轨迹回报	0.8173
回报标准差	0.3797
最小回报	0
最大回报	0.9969
平均轨迹长度	10.11

数据中同时包含失败轨迹和高回报成功轨迹，适合检验模型能否从混合质量离线数据中学习高回报行为。

5.2 主结果：BC、RCBC、短历史 MLP 与 DT

3 个 seed 的正式结果如下。

模型	平均回报 mean	跨 seed 标准差	成功率
BC	0.0000	0.0000	0.00
RCBC	0.0000	0.0000	0.00
SBC-K3	0.6064	0.0359	0.61
SBC-K5	0.6893	0.0152	0.69
DT	0.9938	0.0000	1.00

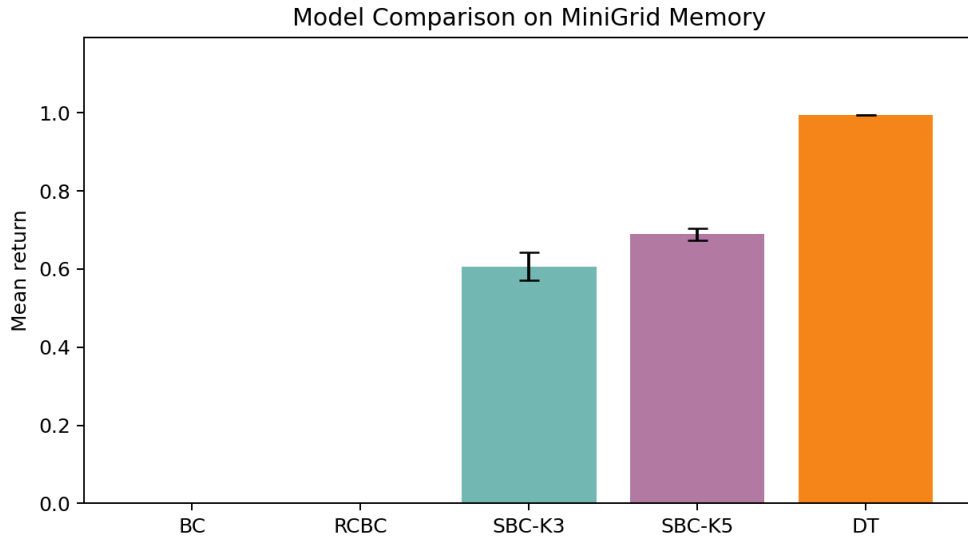


图 1: MiniGrid MemoryS17Random 中各模型的平均回报对比

结果显示, BC 和 RCBC 均无法完成任务; 短历史 MLP 能利用最近几步信息获得部分成功, 但无法稳定完成匹配; DT 在 3 个 seed 上达到 100% 成功率。

5.3 Target Return Sweep

同一个 DT 模型在不同 target return 下的结果如下。

Target return	实际平均回报	跨 seed 标准差	成功率
0.00	0.4407	0.0902	0.4433
0.30	0.5697	0.1357	0.5733
0.95	0.9939	0.0000	1.0000

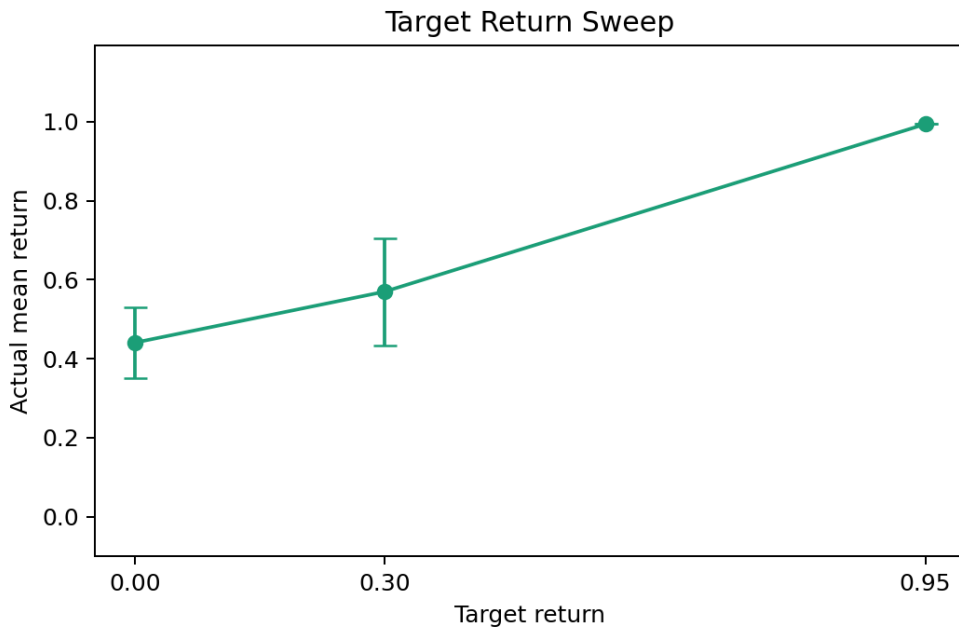


图 2: Decision Transformer 在 MiniGrid MemoryS17Random 中的 target return sweep

该结果表明, target return 越高, 实际回报和成功率越高, 说明模型不仅学到了历史记忆, 也复现了 return

conditioning 的基本效果。

5.4 训练轮数分析

为了避免只报告最终最优结果，本项目额外在 seed 0 上做了 epoch sweep。结果如下。

Epoch	BC 成功率	RCBC 成功率	SBC-K3 成功率	SBC-K5 成功率	DT 成功率
5	0.00	0.00	0.54	0.54	0.45
10	0.00	0.00	0.54	0.57	0.44
20	0.00	0.00	0.58	0.67	0.63
30	0.00	0.00	0.60	0.68	1.00
40	0.00	0.00	0.62	0.71	1.00

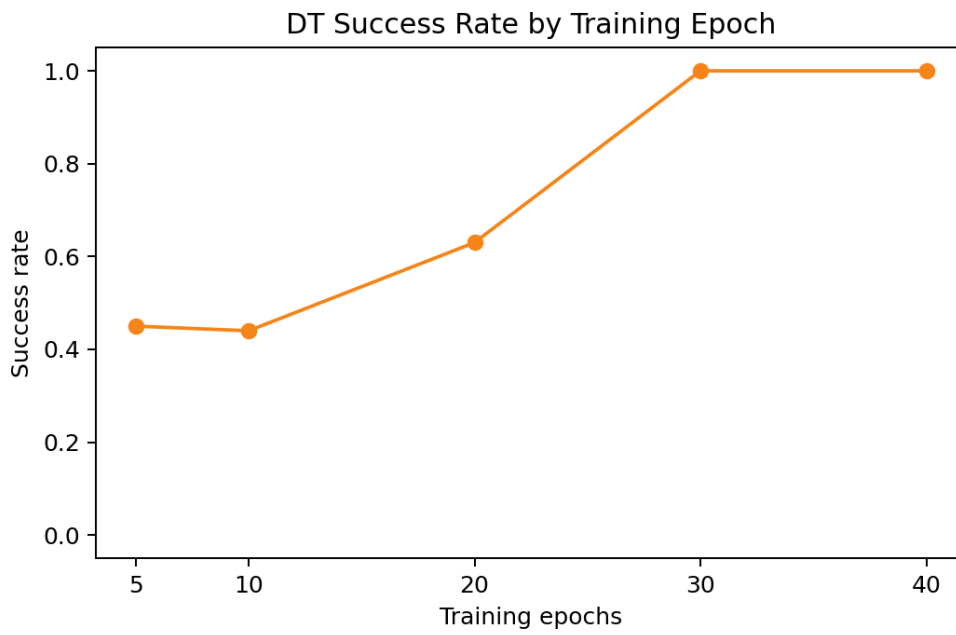


图 3: Decision Transformer 在不同训练轮数下的成功率

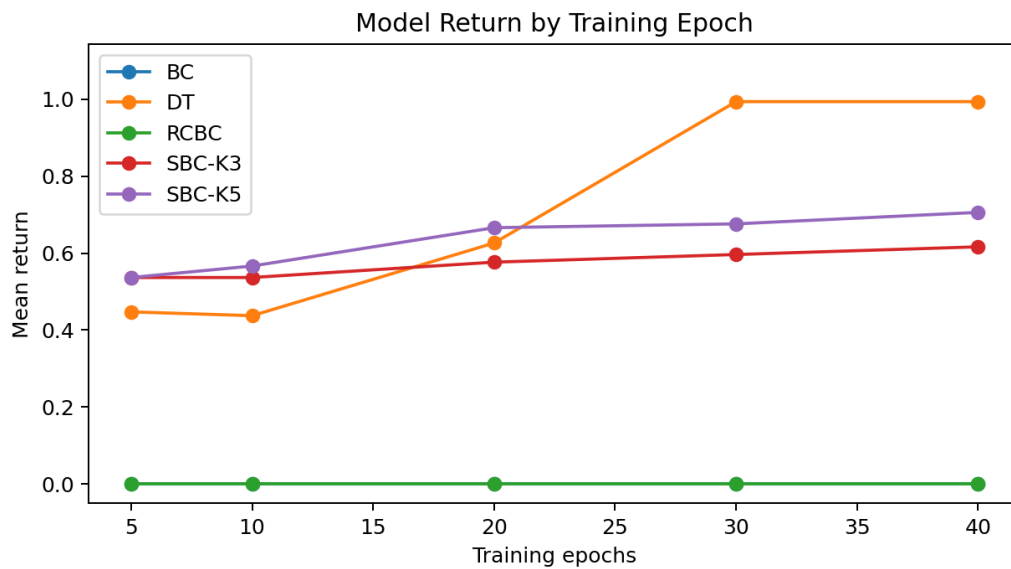


图 4：不同模型随训练轮数变化的平均回报

20 epoch 时 DT 尚未完全收敛；30 epoch 后 DT 达到 1.00 成功率，40 epoch 与 30 epoch 基本一致，因此最终正式实验采用 40 epoch 作为保守配置。

5.5 训练损失

以 seed 0 为例，训练损失曲线如下。

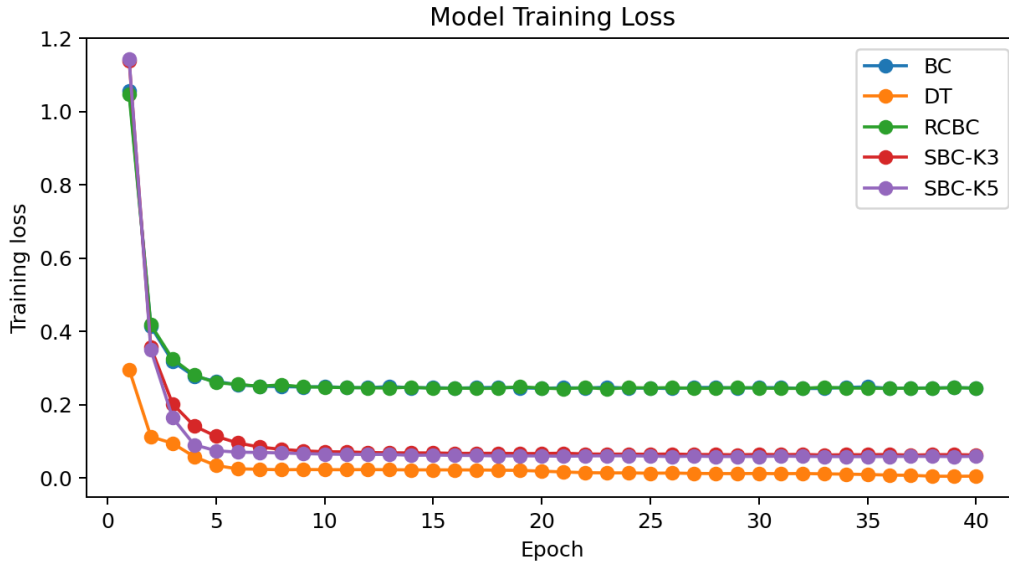


图 5：MiniGrid MemoryS17Random seed 0 的训练损失曲线

损失下降说明模型学到了离线轨迹中的动作预测规律；但最终是否有效仍以环境 rollout 为准。

6 原论文 D4RL 对齐尝试：Hopper

除了课程主实验 MiniGrid Memory，本项目还补做了一个更接近原论文的连续控制 benchmark：mujoco/hopper/medium-v0。这个任务属于 Decision Transformer 原论文的 D4RL OpenAI Gym 实验范围，因此可用于检验 continuous-action DT 管线是否能得到与论文方向一致的趋势。

这里使用七个随机种子 seed0-6 的完整结果。

模型	平均回报 mean	跨 seed 标准差	成功率 *
BC	1899.7	1141.3	0.27
DT	2465.3	696.0	0.34

* 这里的“成功率”不是环境原生成功指标，而是定义为 $\text{return} \geq 3000$ 的比例，只用于辅助观察回报分布。

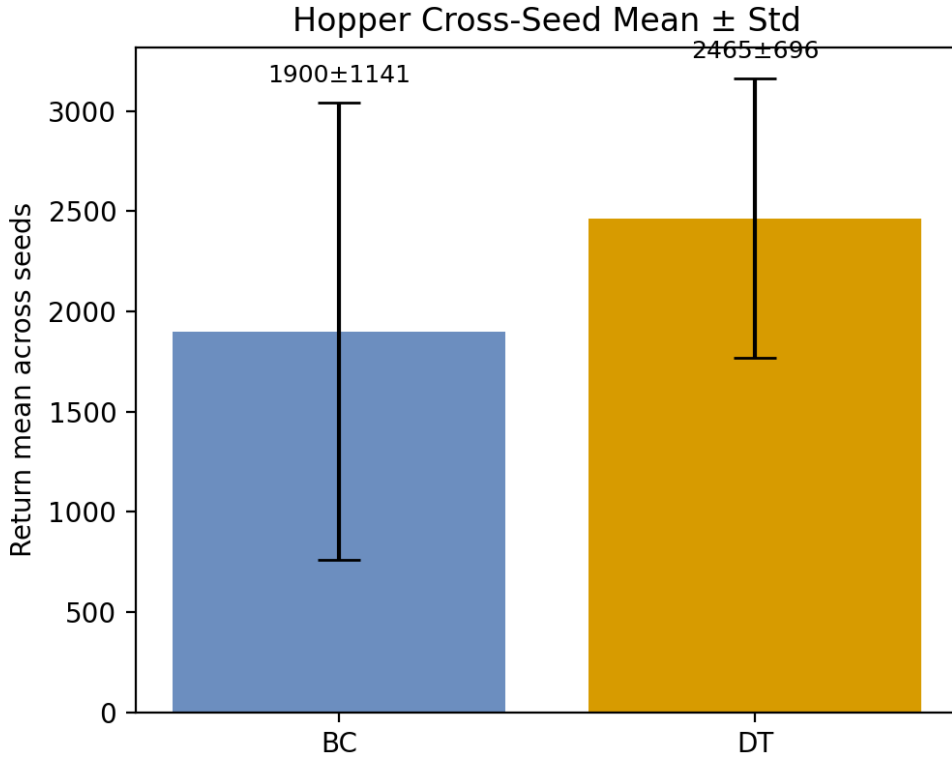


图 6: Hopper-medium 上 BC 与 DT 的跨 seed 均值和误差棒

Hopper 的结果需要克制解读。DT 在平均回报上高于 BC，且跨 seed 标准差更小，说明 return-conditioned sequence modeling 在连续控制任务上出现了与原论文一致的趋势；但由于环境版本与原论文 D4RL 设置并不完全一致，这一部分更适合作为补充 benchmark，而不是统计显著的强结论。

7 结果分析

7.1 为什么 RCBC 失败

RCBC 输入了 return-to-go，因此它是一个公平的条件 baseline。但它只看当前状态，不看历史状态。MemoryS17Random 的关键变量是起点处看到的物体类型；一旦智能体离开起点，当前状态不再包含该信息。虽然 RCBC 在岔路口能看到上下两侧物体，但无法知道要匹配 key 还是 ball，因此不能稳定完成任务。

7.2 为什么 DT 能优于 RCBC

DT 的输入包含最近 K 步的 return-to-go、state 和 action。只要起点物体线索仍在上下文窗口中，Transformer 就有机会通过 causal attention 回看历史信息，并把该信息用于后续动作预测。

实验结果中 DT 平均成功率为 1.00，而 RCBC 为 0，SBC-K5 约为 0.69。这说明在当前课程规模设置下，DT 确实利用了序列历史，把起点 cue 和岔路口物体信息连接起来。

7.3 局限性

本项目仍有明显局限。

第一，MemoryS17Random 的观测经过了手工特征压缩，不是直接从原始图像端到端学习。

第二，Hopper 只是对齐原论文 benchmark 的补充尝试。虽然方向上支持 DT 优于 BC，但样本数有限且环境版本存在偏差，因此只能作为趋势证据。

第三，实验只使用 3 个 seed 作为主实验统计，虽然比单 seed 更好，但仍不足以作为研究论文级结论。

8 结论

本项目完成了 Decision Transformer 的课程版复现。主实验采用 MiniGrid MemoryS17Random 部分可观测记忆任务，结果表明：BC 和 RCBC 平均回报均为 0，SBC-K3 和 SBC-K5 的成功率分别约为 61% 和 69%，而 DT 平均回报为 0.9938、成功率为 100%。这说明在需要记忆的 POMDP 场景中，Decision Transformer 的序列建模能力优于只看当前状态的前馈 baseline，也优于只能利用很短历史窗口的 MLP baseline。

同时，target return sweep 显示目标回报 0、0.3、0.95 分别对应约 0.44、0.57、0.99 的实际回报，说明最终版本也复现了 return conditioning 的基本控制效果。因此，本项目在课程规模下复现了 Decision Transformer 的两个关键机制：通过历史序列解决部分可观测记忆问题，并通过 return-to-go 条件化生成更高回报行为。

项目 GitHub 仓库链接：<https://github.com/chronicar14757/decision-transformer-minigrid-memory>

9 参考文献

- [1] Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. *Decision Transformer: Reinforcement Learning via Sequence Modeling*. Advances in Neural Information Processing Systems, 2021.
- [2] Chevalier-Boisvert, M., Willems, L., and Pal, S. *Minimalistic Gridworld Environment for Reinforcement Learning*. arXiv:2003.05176, 2020.
- [3] Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. *D4RL: Datasets for Deep Data-Driven Reinforcement Learning*. arXiv:2004.07219, 2020.